

ЛАБОРАТОРНАЯ РАБОТА №5: FRONTEND ДЛЯ TODOAPP (SPA И JWT АВТОРИЗАЦИЯ)

Цель работы: Создать клиентскую часть для нашего менеджера задач. Вы научитесь работать с JWT-токенами, сохранять сессию пользователя в браузере и выполнять защищенные запросы к API с использованием Fetch.

КАК МЫ БУДЕМ РАБОТАТЬ:

Вам даны готовые HTML и CSS файлы, чтобы не тратить время на верстку. Ваша главная задача — написать файл `script.js`. Кода JS здесь не будет, вместо этого даны пошаговые инструкции и запросы для гугления.

СПРАВОЧНИК ПО НАШЕМУ АРІ (БЭКЕНД)

Ваш бэкенд ожидает запросы по следующим адресам (предположим, `baseAddress = "http://localhost:5000"`):

Метод	Endpoint	Требует Токен?	Тело запроса (JSON)
POST	<code>/api/v1/Auth/register</code>	Нет	<code>{ "email": "...", "password": "..." }</code>
POST	<code>/api/v1/Auth/login</code>	Нет	<code>{ "email": "...", "password": "..." }</code> <i>Возвращает: { "token": "eyJ..." }</i>
GET	<code>/api/v1/Todos</code>	Да	-
POST	<code>/api/v1/Todos</code>	Да	<code>{ "title": "...", "description": "..." }</code>
PUT	<code>/api/v1/Todos/{id}</code>	Да	<code>{ "title": "...", "description": "...", "isCompleted": true/false }</code>
DELETE	<code>/api/v1/Todos/{id}</code>	Да	-

ШАГ 0: БАЗОВЫЕ ФАЙЛЫ (СКОПИРОВАТЬ)

Создайте файлы `index.html` и `style.css` и вставьте в них следующий код.

index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Мои Задачи</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Секция авторизации -->
  <div id="auth-section" class="container">
    <h2>Вход в систему</h2>
    <form id="auth-form">
      <input type="email" id="email" placeholder="Email" required>
      <input type="password" id="password" placeholder="Пароль" required>
      <button type="submit" id="login-btn">Войти</button>
      <button type="button" id="register-btn">Зарегистрироваться</button>
    </form>
    <p id="auth-error" class="error hidden"></p>
  </div>

  <!-- Секция приложения (скрыта по умолчанию) -->
  <div id="app-section" class="container hidden">
    <div class="header">
      <h2>Мои задачи</h2>
      <button id="logout-btn">Выйти</button>
    </div>

    <form id="todo-form">
      <input type="text" id="todo-title" placeholder="Название задачи" required>
      <input type="text" id="todo-desc" placeholder="Описание">
      <button type="submit">Добавить</button>
    </form>

    <ul id="todo-list"></ul>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f9;
  display: flex;
  justify-content: center;
  padding-top: 50px;
}
.container {
  background: white;
  padding: 20px 30px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  width: 400px;
}
.hidden { display: none !important; }
.error { color: red; font-size: 0.9em; }
input {
  width: 100%; padding: 10px; margin-bottom: 10px;
  border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box;
}
button {
  width: 100%; padding: 10px; margin-bottom: 5px;
  background-color: #007bff; color: white; border: none;
  border-radius: 4px; cursor: pointer;
}
button:hover { background-color: #0056b3; }
#register-btn { background-color: #28a745; }
#logout-btn { background-color: #dc3545; width: auto; padding: 5px 15px; }
.header { display: flex; justify-content: space-between; align-items: center; }
ul { list-style: none; padding: 0; }
li {
  background: #f9f9f9; margin-bottom: 10px; padding: 10px;
  border-radius: 4px; display: flex; justify-content: space-between;
  align-items: center;
}
.completed { text-decoration: line-through; color: gray; }
.todo-actions button { width: auto; padding: 5px; margin: 0 2px; font-size: 0.8em; }
```

ШАГ 1: ИНИЦИАЛИЗАЦИЯ И ПРОВЕРКА ТОКЕНА

Когда пользователь открывает страницу, нам нужно проверить, есть ли у него сохраненный токен. Если есть — показываем задачи. Если нет — форму входа.

1. Создайте файл `script.js`. Объявите константу `const API_URL = "http://localhost:5000/api/v1";` (укажите ваш порт бэкенда).
2. Создайте функцию `checkAuth()`. Внутри неё попытайтесь получить токен из памяти браузера.
3. Если токен есть: скройте `auth-section` (добавьте класс `hidden`) и покажите `app-section` (удалите класс `hidden`). Затем вызовите функцию `loadTodos()` (создадим её позже).
4. Если токена нет: покажите `auth-section` и скройте `app-section`.
5. Вызовите `checkAuth()` в самом низу вашего файла.

Что гуглить: `JS localStorage getItem`, `JS classList add remove`.

Секрет SPA (Single Page Application):

Страница никогда не перезагружается. Мы просто скрываем одни HTML-блоки и показываем другие с помощью CSS-класса `.hidden`.

ШАГ 2: РЕГИСТРАЦИЯ И АВТОРИЗАЦИЯ

Нам нужно оживить форму входа/регистрации.

1. Найдите кнопки `login-btn` и `register-btn` и повесьте на них события `click`. Обязательно используйте `event.preventDefault()` !
2. Соберите значения из полей `email` и `password`.
3. **Для регистрации:** отправьте POST-запрос на `API_URL + "/Auth/register"`. В теле (body) передайте JSON с email и паролем. Если ответ `ok` — выведите `alert("Успешно! Теперь войдите.")`.
4. **Для входа:** отправьте POST-запрос на `API_URL + "/Auth/login"`. Распакуйте JSON-ответ. Если логин успешен, сервер вернет объект `{ token: "ваша_длинная_строка" }`.
5. Сохраните полученный токен в локальную память браузера.
6. Сразу после сохранения токена вызовите функцию `checkAuth()`.

Что гуглить: `JS fetch POST json body headers`, `JS localStorage setItem`.

Как проверить:

Запустите бэкенд и фронтенд. Введите данные, нажмите зарегистрироваться. Затем нажмите войти. Если всё верно, форма входа исчезнет и появится интерфейс задач.

ШАГ 3: ПОЛУЧЕНИЕ ЗАДАЧ (ЗАЩИЩЕННЫЙ МАРШРУТ)

Это самый важный шаг. Бэкенд не отдаст задачи, если вы не приложите к запросу свой "паспорт" — JWT токен.

1. Создайте асинхронную функцию `loadTodos()`.
2. Достаньте токен из `localStorage`.
3. Сделайте GET запрос на `API_URL + "/Todos"`.
4. **ГЛАВНАЯ МАГИЯ:** В настройках `fetch` добавьте объект `headers`. В него добавьте заголовок `"Authorization": "Bearer " + токен` (обратите внимание на пробел после слова Bearer).
5. Распакуйте ответ в массив задач и передайте его в функцию `renderTodos(tasks)` (напишем её ниже).

Что гуглить: `JS fetch Authorization Bearer header`.

ШАГ 4: ОТРИСОВКА ЗАДАЧ (RENDER)

1. Создайте функцию `renderTodos(tasks)`.
 2. Очистите содержимое `#todo-list` (`innerHTML = ''`).
 3. Пройдитесь циклом (например, `forEach`) по массиву `tasks`.
 4. Для каждой задачи создайте элемент `<li>`. Если задача выполнена (свойство `isCompleted`), добавьте `li` класс `"completed"`.
 5. Внутри `li` вставьте HTML-код с текстом задачи и двумя кнопками: "V" (выполнить) и "X" (удалить). Подсказка: чтобы кнопки знали, к какой задаче они относятся, добавьте в их тег `onclick="deleteTodo(${task.id})"` и `onclick="toggleTodo(${task.id}, ${task.isCompleted})"`.
 6. Добавьте `li` в `#todo-list`.
-

ШАГ 5: ДОБАВЛЕНИЕ НОВОЙ ЗАДАЧИ

1. Повесьте слушатель на `submit` формы `#todo-form` (не забудьте `preventDefault()`).
 2. Возьмите значения из полей `#todo-title` и `#todo-desc` .
 3. Отправьте POST-запрос на `API_URL + "/Todos"` . Обязательно передайте:
 - Тело запроса (title и description в JSON)
 - Заголовок `Content-Type: application/json`
 - Заголовок `Authorization: Bearer + токен`
 4. После успешного ответа очистите форму ввода и заново вызовите `loadTodos()` , чтобы список обновился.
-

ШАГ 6: УДАЛЕНИЕ И ОБНОВЛЕНИЕ ЗАДАЧ

1. Создайте функцию `deleteTodo(id)`. Она должна отправлять `DELETE` запрос на `API_URL + "/Todos/" + id` (с токеном в заголовках!). После успеха вызовите `loadTodos()`.
2. Создайте функцию `toggleTodo(id, currentStatus)`. Она должна отправлять `PUT` запрос на `API_URL + "/Todos/" + id`. В теле запроса отправьте объект, где `isCompleted` равно `!currentStatus` (противоположное значение). После успеха вызовите `loadTodos()`.

(Внимание: PUT запрос требует отправки title и description тоже, согласно API бэкенда. Вам нужно будет либо получить текущую задачу из массива, либо бэкенд должен позволять частичное обновление).

ШАГ 7: ВЫХОД ИЗ СИСТЕМЫ (LOGOUT)

Выход из системы при использовании JWT — это просто удаление токена из памяти.

1. Найдите кнопку `#logout-btn` и повесьте на неё `click`.
2. При клике удалите токен из памяти браузера.
3. Вызовите функцию `checkAuth()` (которая сама поймет, что токена нет, скроет задачи и покажет форму входа).

Что гуглить: `JS localStorage.removeItem`.

ФИНАЛЬНАЯ ПРОВЕРКА:

Вы должны иметь возможность зарегистрировать пользователя, войти, добавить задачу, отметить её выполненной, удалить её, нажать "Выйти" и снова увидеть пустую форму входа. Если вы обновите страницу (F5) будучи залогиненным, вы не должны "вылетать" из системы.